

RePrint — Product Requirements Document

Sep 28, 2026 · @Preston · Status: Draft v2

1. Overview

RePrint is a web app for discovering books and reading and writing trustworthy, moderated reviews. Every review is checked by a moderator before it is published. Personal libraries are a secondary, semi-social feature that gives readers a reason to create an account and come back.

- **Product:** RePrint
- **Platform:** Web only, responsive from phone to desktop. Server-rendered so book pages can be found through search engines and shared on social media.
- **Domains:** `www.reprint.com` (web app) and `api.reprint.com` (API).
- **Release:** One public launch (v1) with every feature in this document. Work is split into internal milestones (section 3), but nothing ships publicly until all of them are done.
- **Replaces:** The December 2025 RePrint MVP PRD. That draft mixed Hardcover and Open Library, left out account recovery, and excluded testing and CI. This version fixes all three.

Product principles

1. **Discovery and reviews come first.** Every page should help someone find a book or judge one. Libraries support this; they don't compete with it.
2. **Trust over volume.** Each review is tied to a verified account, limited to one per user per book, and approved by a moderator.
3. **RePrint owns its data model.** Outside book sources are inputs, translated into RePrint's own vocabulary (sections 5 and 6). No provider's terms or IDs leak into the product.
4. **Production-ready from day one.** Security, account recovery, testing, CI/CD, monitoring, and accessibility are part of v1, not a later phase.

2. Goals and success metrics

v1 succeeds when readers can find a book, see what verified RePrint users think of it, and add their own review, with moderation keeping quality high. Targets will be set after launch, once there is a baseline. At launch, we will track the following:

Area	Metric	Why it matters
Discovery	Book page views per visit; searches that lead to a book page click	Measures whether discovery works
Reviews	Reviews submitted per week; approval rate; median time from submission to decision	Measures review volume and moderation health
Trust	Reports per 1,000 approved reviews; share of reviews marked helpful	Measures review quality
Engagement	Weekly active users; share of new accounts that add a library book or write a review within 7 days	Measures whether libraries bring people back
Growth	Organic search traffic to book pages	Validates the server-rendering decision
Reliability	API uptime; p95 latency; book source error rate	Measures production health

Metrics come from privacy-friendly analytics (section 11). No third-party ad trackers.

3. Scope

v1 launches only when all eight milestones below are complete. The milestones set the build order and give us checkpoints to review work; none of them is a public release.

Build milestones

1. **Foundation:** monorepo, CI pipeline, database and migrations, environments, error tracking, and design system setup.
2. **Accounts:** registration, email verification, login and logout, sessions, password reset, account settings, account deletion, and rate limiting.

3. **Book catalog:** RePrint domain model, provider interface, Open Library adapter, search, book pages, author pages, and caching.
4. **Reviews:** write, edit, and delete reviews; moderation queue; rejection reasons; ratings and rating distribution; spoiler handling.
5. **Discovery:** Discover home page, Genre and Series browsing, rankings, helpful votes, and review sorting.
6. **Libraries and profiles:** shelves, library pages, and public profiles.
7. **Trust and admin:** review reports, user management, roles, suspensions, and audit log.
8. **Launch readiness:** SEO (sitemap, structured data), accessibility audit, load test, security review, legal pages, backups, and a restore drill.

Out of scope for v1

- Following users, activity feeds, and comments on reviews
- Logging in with Google, Apple, or other third-party accounts; multi-factor authentication (planned; the data model leaves room for it)
- Custom shelves beyond the three statuses; reading progress and dates
- Native mobile apps
- Automatic approval of reviews. Every review is always moderated.
- A second book data provider in production. The architecture supports one (section 6), but v1 ships with Open Library only.
- Languages other than English. User-facing text is kept in one place so translation can be added later.
- Monetization, ads, and affiliate links

4. Users, roles and permissions

Access is controlled by **permissions**, grouped into **roles**. A user can have more than one role. The app always checks permissions, never role names, so adding a new role later only means adding data, not changing code.

Roles at launch

- **Visitor:** not signed in.
- **Member:** a signed-in user. Everyone who registers gets this role.
- **Moderator:** reviews content (reviews and reports).
- **Admin:** full access, including user and role management.

The first Admin is created by a one-time command run on the server (`pnpm --filter api seed:admin`). All later role changes happen in the Admin area.

Permission	Visitor	Member	Moderator	Admin
Search, browse, view books, approved reviews, public profiles	Yes	Yes	Yes	Yes
Write, edit, delete own review (verified email required)	No	Yes	Yes	Yes
Vote a review helpful; report a review	No	Yes	Yes	Yes
Manage own library and profile	No	Yes	Yes	Yes
<code>reviews.moderate</code> : approve, reject, and unpublish reviews	No	No	Yes	Yes
<code>reports.resolve</code> : handle review reports	No	No	Yes	Yes
<code>users.view</code> : search users, view account details	No	No	Yes (limited)	Yes
<code>users.suspend</code> : suspend, unsuspend, force logout	No	No	No	Yes
<code>roles.assign</code> : grant or remove roles	No	No	No	Yes
<code>audit.view</code> : read the audit log	No	No	No	Yes

Rules

- Moderators cannot moderate their own reviews.
- Admins cannot remove the Admin role from themselves if they are the last Admin.
- A suspended user cannot log in. Their existing sessions end immediately. Their approved reviews stay visible unless a moderator unpublishes them.
- Every action taken with an elevated permission (Moderator or Admin) is recorded in the audit log.

5. Domain vocabulary

RePrint has one vocabulary, and it is the only one used in the UI, API, database, and code outside the Source adapters. Every Source is translated into these terms when its data arrives, so its own words (Open Library's "work," Google Books' "volume," Hardcover's "contribution") never leave its adapter. The terms are split into three groups: the Catalog (what books are), provenance (where the data came from), and community (what Members do).

5.1 Catalog terms

Term	Meaning	Must have	May have
Book	A title as a reader thinks of it, across every printing, format, and translation. Reviews, ratings, shelves, and helpful votes attach here.	Title; at least one Author Contribution; a Source link or an ISBN-13	Subtitle, description, first published year, original language, Primary Edition, Cover, Series memberships, Genres, Subjects
Edition	One published version of a Book.	The Book it belongs to; a Source link	ISBN-13 (ISBN-10 is always converted to 13), Format, Language, publisher name, publication date, page count, Cover
Primary Edition	The Edition whose cover and details represent the Book by default.	—	Chosen automatically (English, has a cover, has an ISBN, most recent), or set by an admin.
Author	A person credited on one or more Books. Has their own page.	Name	Alternate names, bio, birth and death dates, photo

Term	Meaning	Must have	May have
Contribution	Links an Author to a Book with a Role : Author, Co-author, Translator, Illustrator, Editor, Narrator, or Other.	Author, Book, Role	Position (the order of names in the byline)
Series	A named, ordered set of Books, such as <i>The Expanse</i> .	Name	Description; each Book's position, which can be a decimal (2.5 for a novella) or empty
Genre	A browse category from RePrint's own curated list (about 40, managed by admins), for example Science Fiction or Memoir. This is what Discover, filters, and genre pages use.	Name, slug	Description, parent Genre
Subject	A raw topic tag supplied by a Source, for example "Space warfare" or "Fiction, general." It is used to improve search and is mapped to Genres. It is never shown as a browse category.	Label	Mapped Genre
Cover	A cover image with its origin and size variants.	Image location, origin	Width, height
Format	Hardcover, Paperback, Ebook, Audiobook, or Unknown.	—	—

Term	Meaning	Must have	May have
Language	An ISO 639 code stored on each Edition. The Book's original language is optional.	—	—

Translations are Editions of the same Book, so reviews of *Cien años de soledad* and *One Hundred Years of Solitude* land in the same place.

5.2 Provenance terms

Term	Meaning
Catalog	RePrint's own store of Books, Editions, Authors, Series, Genres, and Covers. Every page is served from it.
Source	An outside provider of book data, reached through an adapter.
Source link	Connects a Catalog record to its ID at a Source. One Catalog record can have links to several Sources.
Source record	The raw response from a Source, kept for 30 days to help with debugging.
Field origin	For each field on a Catalog record: which Source set it, or "admin" if a person did, and when. Fields set by an admin are locked.
Storage policy	What a Source's terms allow: Store (keep permanently in the Catalog), Cache (keep only as long as the terms allow, never in the Catalog), or None (not usable).

5.3 Community terms

Term	Meaning
Member	A registered account. In code this is <code>User</code> .
Review	One Member's rating (1 to 5 whole stars) and written text about a Book. There is at most one per Member per Book. A rating is always part of a review; there are no rating-only entries.
Review status	Pending, Approved, Rejected, or Unpublished.
Review version	One submitted version of a Review, along with the moderation decision made on it.
Helpful vote	A Member marking someone else's Approved Review as helpful.
Report	A Member flagging an Approved Review for a moderator to check.
Shelf	Want to Read, Reading, or Read.
Shelf entry	A Book on a Member's Shelf. There is at most one per Member per Book.
Library	All of a Member's Shelf entries.

5.4 Rules

- **IDs:** Every Catalog record gets a RePrint ID (UUIDv7) and a readable slug, for example `/books/the-left-hand-of-darkness-0192a3`. A Source's ID never appears in a URL, API response, or foreign key.
- **Shared types:** Domain types are defined once as Zod schemas in `packages/shared`. Adapters must return exactly these types. Anything that fails validation is logged and skipped, never stored.
- **Missing data is allowed:** Any field not marked "Must have" can be empty. The UI has a defined fallback for each: a generated cover showing the title and author, "No description yet," and hidden rows for missing Edition details.
- **Matching:** A new Source record is linked to an existing Catalog record, instead of creating a new one, if one of these matches, checked in order:

1. An existing Source link.
2. An ISBN-13 on any of its Editions.
3. An identifier the Source shares with the Catalog, such as Wikidata IDs for Authors.

A match on title and author alone is never merged automatically. It is added to the admin merge queue instead.

- **Genres come from mapping:** Subjects are mapped to Genres through a table of patterns kept in the Catalog admin. Admins can add or remove a Genre on any Book, and those edits are locked like other admin fields.
- **Ownership:** Reviews, ratings, shelves, and helpful votes are RePrint data. They are never sent to any Source.

6. Book data sources

RePrint stores only the Books its Members actually use, in its own **Catalog** in Postgres. Everything else is looked up live from a Source when someone searches.

- **Pages are always served from the Catalog.** This includes book, author, genre, series, and Discover pages, and anything with reviews or shelves. They never wait on a Source.
- **Search reaches Sources live.** Results are cached, so the same search doesn't repeatedly hit a Source.
- **No pre-launch import.** The Catalog starts empty (apart from local sample data) and grows with use.

This keeps storage small. Neon charges [\\$0.35 per GB-month](#), and at about 10 KB per stored Book, 100,000 Books is about 1 GB. It also respects Open Library's [API guidelines](#), which allow 3 requests per second for identified apps and ask developers not to use Open Library as the backend for high-traffic services.

What a Source must provide

A Source can fill the Catalog only if it meets all four **required** needs. **Preferred** needs decide which fields it's trusted for.

- **Required:**
 - Terms that allow permanent storage (storage policy Store).
 - Editions grouped into Books, or ISBNs we can group by.
 - Search by title, author, and ISBN.
 - Title and author names on every record.

- **Preferred:** Author records with bios and photos; contributor roles; Series; subjects or genres; descriptions; covers; complete Edition details; bulk download; stable API; free.

How the candidates compare (checked September 28, 2026)

Need	Open Library	Google Books	Hardcover
Terms allow storing in the Catalog (required)	Yes. Catalog data is CC0 public domain (FAQ)	No. Google's API terms bar building databases or keeping copies beyond the cache header (terms)	Unconfirmed. Rules cover user data; aggregate data is allowed (docs)
Editions grouped into Books (required)	Yes (works)	No, each result is one edition	Yes
Search (required)	Yes	Yes	Yes (GraphQL)
Author records	Yes: bio, dates, photos, cross-IDs including Wikidata	Names only	Yes
Contributor roles	Partial	No	Yes
Series	Partial: on some works and editions	No	Yes
Subjects or genres	Many subjects, uneven quality	A few broad categories	Genres and tags
Descriptions	On many works, uneven	Usually present	Usually present
Covers	Yes. Linking to their image server is encouraged; crawling it is not (covers)	Yes	Yes (user uploads need a takedown process)
Edition details (ISBN, format, pages, language)	Present but uneven	Yes	Yes

Need	Open Library	Google Books	Hardcover
Bulk download	Monthly dumps: works about 2.9 GB, editions about 9.2 GB, authors about 0.5 GB (dumps)	None	None
Rate limits	3 per second when identified (API)	Daily quota per API key	60 per minute, 5,000 per day (free tier)
API stability	Long-running and stable	Stable	Beta: breaking changes and token resets are possible

Conclusion

- **Open Library** meets every required need. It stays the primary Source.
- Its weak spots are covered as follows:
 - Series and contributor roles are only partial. Admins can edit them, and a second Source can fill them in later.
 - Subjects are noisy. They are mapped to curated Genres (section 5.4).
 - Descriptions and Edition details are uneven. The UI has fallbacks for missing fields.
- **Hardcover** is the best second Source for series, roles, genres, and descriptions, but only if Hardcover confirms in writing that we can store its data. Until then it stays out of v1.
- **Google Books** can't fill the Catalog under its terms. At most, a book page could show a plain "View on Google Books" link.

Provider interface

Each Source is an adapter in `apps/api/src/catalog/sources/<name>/` that implements one interface:

- `searchBooks(query, page)` : returns **Book candidates**. A Book candidate is a RePrint Book with at least one Edition, the Source link, and a match confidence from 0 to 1.
 - A Source that works at the Book level (Open Library works, Hardcover books) maps each result to one candidate.
 - A Source that works at the Edition level groups its results into candidates by ISBN-13, then by normalized title and first author.

- `getBook(sourceId)` , `getEditions(sourceId)` , `getAuthor(sourceId)` : return full records for storing a Book in the Catalog.
- `importBulk(stream)` : optional and not used in v1. It's there so a bulk import can be added later if live search traffic outgrows a Source's rate limit.

Adapters return RePrint domain types (section 5) and never raw provider data. The Catalog keeps only the trimmed fields RePrint displays. Raw responses are kept in `source_records` for 30 days to help with debugging, then deleted.

Each adapter also declares its **storage policy** and the fields it is trusted to fill. The Catalog accepts records only from Store Sources. When two Sources supply the same field, a per-field priority list decides which one wins. For example: description from Open Library, falling back to a second Source. Admin edits always win.

When a Book is added to the Catalog

A Book is stored the first time one of these happens:

- Someone opens its book page.
- A Member shelves it.
- A Member reviews it.

When someone opens a search result that isn't in the Catalog yet, the API fetches the full record. That takes about a second, with a 5-second limit. It stores the Book with its Editions and Authors, then redirects to the new `/books/<slug>` page. If the Source doesn't respond, the page shows "We couldn't load this book right now," with a retry button.

Keeping it current

- When a stored Book is viewed and its data is more than 30 days old, a background job re-fetches it.
- Admins can refresh any Book on demand.
- Admin edits are locked, so a refresh never overwrites them.

Growth

The Catalog only grows; Books are never removed, since reviews and shelves point at them. An admin dashboard shows the Catalog's size and monthly growth.

How search works

1. The query runs against the Catalog: Postgres full-text and trigram search across Book titles, Edition titles, ISBNs, Author names, and Series names.

2. At the same time, the API asks the Source for page 1 of results. Source results are cached in Redis for 24 hours, keyed on the normalized query and page number.
3. Source candidates are matched against the Catalog using the matching rules in section 5.4. A candidate that matches is shown as the stored Book, with its RePrint rating. The rest are shown as Books "not yet on RePrint."
4. Results are merged and ranked:
 - Catalog Books get a relevance boost based on review count.
 - Exact ISBN matches go first.
 - Each Book appears only once.
5. If the Source is slow (no answer within 1.5 seconds) or down, Catalog results are shown alone, with a note that more results may be available later.

Suggestions while typing come only from the Catalog, so keystrokes never reach a Source. Pressing Enter runs the full search above.

Pagination: Page 1 combines Catalog and Source results. Later pages ask the Source for its matching page and remove Books already shown.

Protecting the Source

- **Rate limit:** All outgoing requests, from search, Book fetches, and refreshes, pass through one shared Redis rate limiter set to 2 requests per second. Interactive requests (search and first Book views) get priority over background refreshes.
- **Circuit breaker:** Opens after repeated errors. While it's open, search shows Catalog results only.
- **Identification:** Every request sends `User-Agent: RePrint/<version>(ops@reprint.com)`.
- **If the Source is down:**
 - Every stored Book, review, profile, and Discover page keeps working normally.
 - Search shows only Catalog results.
 - Opening a Book that isn't stored yet shows the retry message.
- **Watching headroom:** A dashboard tracks Source requests per second and the search cache hit rate. An alert fires if usage stays above 70% of the rate limit for an hour. That is the signal to consider a bulk import or a second Source.

Covers

Book covers stay on the Source's image server, since Open Library encourages linking to it. We request covers by cover ID, which isn't rate-limited, and never crawl. A Cover record stores its origin and sizes, so it can point to a different host later without code changes. If an image is missing or fails to load, a generated cover showing the title and author is displayed instead. Admin-uploaded covers and Member avatars go to our own storage (Cloudflare R2 behind a CDN). The site footer credits Open Library.

Adding a Source later (Hardcover is the leading candidate):

1. Confirm its terms allow storage.
2. Write an adapter that passes the shared contract test suite (section 12).
3. Set the adapter's field priorities.
4. Backfill: records are matched to the Catalog using the matching rules in section 5.4.

No schema changes are needed.

7. Features

This section covers every screen and flow in v1, in roughly the order a reader encounters them.

7.1 Accounts and sign-in

- **Register:** The user enters an email, a username, and a password.
 - Username: 3 to 30 characters (letters, numbers, or underscores), unique regardless of case.
 - Password: at least 12 characters. It is checked against known breached passwords using the Have I Been Pwned range API, which never sends the full password.
- **Verify email:** A single-use link, valid for 24 hours, is sent at registration. Unverified accounts can browse and shelve Books, but cannot write reviews, vote, or report until they verify. A banner offers to resend the link.
- **Log in:** Email and password. A failed attempt always shows the same generic message, so it doesn't reveal whether an email is registered. Attempts are rate-limited per IP and per account (section 11).
- **Sessions:**
 - A session stays valid for 30 days and renews while in use.
 - In settings, users see a list of active devices and can end any of them, or log out everywhere.

- **Password reset:** "Forgot password" sends a single-use link valid for 1 hour. The response is identical whether or not the email exists. Resetting the password ends all sessions.
- **Account settings:**
 - Changing the email requires the current password, and the new address must be verified before it takes effect.
 - Changing the password requires the current password and ends all other sessions.
 - Users can also change their display name, bio, avatar, library privacy, and email notification preferences.
- **Delete account:** Requires the password. The account is disabled immediately and permanently erased after 30 days. The user's reviews, votes, reports, and library are deleted along with it.

7.2 Discover (home page)

The home page is built for browsing, not for account holders. Visitors see the same page as members, apart from sign-in prompts. It has these rows:

- **Recently reviewed:** Books with the newest approved reviews, one card per Book.
- **Top rated on RePrint:** Ranked by weighted average (section 7.6). A Book needs at least 5 approved reviews to appear.
- **Most reviewed this month:** Ranked by approved reviews in the last 30 days.
- **Browse by genre:** 12 featured Genres, chosen by admins, plus a link to all Genres.
- **Featured review:** A single approved review chosen by a moderator, shown with its Book.

Rows are cached and rebuilt every 10 minutes. The Catalog starts empty, so a row with fewer than 6 Books is hidden until it fills.

7.3 Search

- **One search box** in the header on every page. After 2 characters, it suggests Books and Authors from the Catalog, waiting 250 ms after the last keystroke.
- **Full search** combines Catalog results with live Source results, as described in section 6 ("How search works").
- **Results page:**
 - Tabs for Books and Authors.

- Each Book shows its cover, title, authors, first published year, and, if it's stored, its RePrint average rating and review count.
- Books not yet on RePrint are shown the same way, with "No RePrint reviews yet."
- **Filters:**
 - Genre, language, and minimum rating apply to Catalog Books. Source results don't have RePrint Genres or ratings, so choosing one of these filters limits results to the Catalog.
 - First-published decade applies to all results.
- **Sort:** Relevance (default), most reviewed, highest rated, or newest.
- **ISBN search:** Entering a 10- or 13-digit ISBN goes straight to that Book, fetching it from the Source if needed.
- **Pagination:** 20 results per page. The page number is in the URL, so results can be shared.

7.4 Book page

URL: `/books/<slug>`. The page is server-rendered.

- **Header:** Cover, title and subtitle, and contributors with their roles (for example, "translated by"). Also the Series name and position with a link, first published year, page count, publisher, and Genre tags. Edition details come from the Primary Edition.
- **Description:** Shown in full, collapsed after 6 lines.
- **Rating summary:** Average (one decimal), count, and a 5-bar distribution chart. Clicking a bar filters the review list to that star rating.
- **My controls** (signed in): a shelf selector, and either "Write a review" or the member's own review with its status.
- **Reviews:**
 - Approved reviews only.
 - Sort by Most helpful (default), Newest, Highest, or Lowest. Filter by star rating.
 - 10 per page.
- **Editions:** A collapsible list of known Editions with format, year, publisher, and ISBN.
- **More by this author:** Up to 6 Books.
- **SEO:** schema.org `Book` data with `AggregateRating` and `Review` entries, plus Open Graph tags for link previews.

7.5 Author, Genre, and Series pages

- **Author** (/authors/<slug>): Photo, name, life dates, and bio when available. Lists the Books they contributed to, grouped by role (written, translated, narrated, and so on) and sorted by RePrint review count, with each Book's rating.
- **Genre** (/genres/<slug>): Books in that Genre and its child Genres. Sort by top rated, most reviewed, or newest review.
- **Series** (/series/<slug>): Books in reading order, each with its position, RePrint rating, and the viewer's shelf status.

7.6 Reviews

Writing a review

- **Rating:** 1 to 5 whole stars. Required.
- **Headline:** Up to 120 characters. Optional.
- **Body:** 50 to 10,000 characters. Required. Written as plain text with paragraphs; links are shown as text and are not clickable.
- **Contains spoilers:** A checkbox. When checked, the review body is hidden behind a "Show spoilers" button.
- **Edition read:** Optional. For example, "I listened to the audiobook."

Rules

- **One per Book:** Each Member can write one review per Book. This is enforced by a unique database constraint on (user, Book).
- **Moderation:** Every new or edited review goes to Pending. Editing an Approved review sends it back to Pending, and it is hidden until approved again.
- **Rejection:** A rejected review stays visible only to its author, along with the moderator's reason if one was given. The author can edit and resubmit it.
- **Deletion:** An author can delete their own review at any time. The deletion is permanent.
- **Notifications:** The author gets an in-app notification and an email (if enabled in settings) when their review is approved, rejected, or unpublished.
- **Averages:** Average and count use Approved reviews only. Rankings use a weighted average, $(C \times m + \sum \text{ratings}) / (C + n)$, where m is the site-wide average rating, C is 5, and n is the Book's review count. This keeps a Book with one 5-star review from topping the list.

Helpful votes

- Any verified Member can mark an Approved review as helpful, once, and can remove the vote. Members cannot vote on their own reviews.
- Reviews show "N people found this helpful" and are sorted by vote count, with ties broken by newest first.

7.7 Libraries

- **Shelves:** Want to Read, Reading, and Read. A Book is on at most one shelf per Member, and choosing a new shelf replaces the old one.
- **Where to shelve:** From the Book page, search results, and Discover cards, using one control that shows the current shelf. Choosing "Remove" takes the Book off the shelf.
- **Library page** (`/u/<username>/library`): Tabs for All, Reading, Want to Read, and Read, with counts. Sort by date added (newest or oldest), title (A to Z), or author.
- **Privacy:** Public by default, visible on the profile. Members can make their library private in settings.
- **Independent of reviews:** Shelving a Book is never required to review it, and reviewing a Book never shelves it.

7.8 Public profiles

`/u/<username>` shows:

- Avatar, display name, username, bio (up to 280 characters), and join date.
- Totals: approved reviews and total helpful votes received.
- Tabs: Reviews (Approved only, newest first) and Library (if public).

Avatars are uploaded, re-encoded to WebP at 256 px, and stored in R2.

7.9 Reporting a review

- Verified Members can report an Approved review they didn't write, once per review. The reason is one of: unmarked spoiler, offensive or hateful, spam or advertising, off-topic, or other (with a note of up to 500 characters).
- Reports go into the moderation Reports queue, grouped by review.
- A review with 3 or more open reports is automatically hidden until a moderator decides. This is the only moderation action the system takes on its own.

7.10 Moderation

Review queue (`/admin/reviews`): Pending reviews, oldest first. Each item shows:

- Book, reviewer, rating, full text, and spoiler flag.
- The reviewer's history: approved, rejected, and reported counts.
- For an edited review, a side-by-side comparison with the last approved version.

Actions:

- **Approve** or **Reject** (with an optional reason, chosen from saved phrases or typed).
- Keyboard shortcuts: `A` to approve, `R` to reject, `J / K` for next and previous.
- Opening an item claims it for 10 minutes, so two moderators don't handle the same review.

Reports queue (`/admin/reports`): Each item shows the review, the reports on it, and the reasons given. Actions:

- **Dismiss**: Keeps the review visible and closes the reports.
- **Unpublish**: Moves the review to Unpublished, with a reason. The author is notified and can edit and resubmit.
- **Suspend author**: Admins only.

Moderation dashboard: Counts of pending reviews and open reports, plus the age of the oldest item in each queue.

7.11 Administration

- **Users** (`/admin/users`):
 - Search by email or username. Filter by role, status (active, unverified, suspended, deleted), or join date.
 - The user detail page shows the profile, roles, sessions, reviews by status, reports filed and received, and audit history.
 - Actions: assign or remove roles; suspend with a reason and an optional end date; unsuspend; end all sessions; resend verification email.
- **Catalog**: Edit a Book's title, description, cover, Genres, Series, and contributions; edited fields are locked. Refresh a Book from its Source. Work through the merge queue and merge duplicate Books. A merge moves reviews and shelves to the remaining Book, and fails if one Member reviewed both. Manage the Genre list and the rules that map Subjects to Genres.

- **Featured content:** Choose the Discover page's featured Genres and featured review.
- **Audit log** (`/admin/audit`): Every elevated action, recorded as who, what, target, before and after values, time, and IP. It can be filtered and exported to CSV. It is append-only and cannot be edited.

7.12 Email and notifications

- **In-app notifications:** A bell icon with an unread count. Covers review approved, rejected, or unpublished, and account security events.
- **Transactional emails:** Verify email, password reset, email change (sent to both the old and new address), password changed, review decision (optional, on by default), account suspended, and account deletion scheduled.
- **Delivery:** Resend. Email templates are built with React Email in `packages/email`.

7.13 Legal and static pages

Terms of Service, Privacy Policy, Community Guidelines (what gets a review rejected), About, and Contact. A cookie notice is only needed if analytics sets cookies; the analytics we use does not.

8. Technical architecture and stack

RePrint is a TypeScript monorepo with two deployable apps: a server-rendered web app and a separate API. Validation schemas are shared between them. Versions below are the latest stable releases on npm as of September 28, 2026. We pin major versions and let Renovate open pull requests for updates.

Repository layout (pnpm workspaces + Turborepo)

<code>apps/web</code>	React Router framework app, server-rendered (<code>www.reprint.com</code>)
<code>apps/api</code>	Fastify API + background worker entry point (<code>api.reprint.com</code>)
<code>packages/shared</code>	Zod schemas, domain types, permission names, constants
<code>packages/db</code>	Drizzle schema, migrations, seed scripts
<code>packages/email</code>	React Email templates
<code>packages/ui</code>	shadcn/ui components + Tailwind theme
<code>packages/config</code>	Shared tsconfig and Biome settings

Stack

Layer	Choice	Version	Notes
Runtime	Node.js	24 LTS	Same version everywhere: CI, Render, and Netlify functions
Language	TypeScript	7.0	The new native compiler. If a tool still needs the old compiler API, pin <code>typescript@6</code> for that tool only
Package manager and build orchestration	pnpm, Turborepo	12.6, 2.11	Turborepo caches builds and tests
Web framework	React Router (framework mode)	8.4	Server rendering, loaders and actions
UI library	React	19.3	
Build tool	Vite	8.3	
Styling	Tailwind CSS, shadcn/ui	4.3, CLI 4.21	Dark theme with the original palette: <code>#0f172a</code> / <code>#3b82f6</code> / <code>#f8fafc</code>
Client data fetching	TanStack Query	5.104	Only for client-side changes and updates after the page loads; loaders handle the first page load
Forms	React Hook Form, <code>@hookform/resolvers</code>	7.89, 5.9	Uses the same Zod schemas as the API
Validation	Zod	4.6	Defined once in <code>packages/shared</code>
API framework	Fastify	5.12	Plus <code>@fastify/cookie</code> , <code>cors</code> , <code>helmet</code> , <code>rate-limit</code>

Layer	Choice	Version	Notes
Database	PostgreSQL on Neon	18 (confirm Neon support at setup; 17 as fallback)	Extensions: <code>pg_trgm</code> , <code>unaccent</code>
Database toolkit	Drizzle ORM, drizzle-kit	0.45, 0.31	Drizzle 1.0 is at release candidate 5; upgrade once it is stable
Postgres driver	postgres.js	3.4	
Cache, rate limits, job queue	Redis (Render Key Value) with BullMQ, ioredis	6.3, 6.0	Background jobs: Book refreshes, emails, sitemaps, Discover rows
Password hashing	Argon2id via <code>@node-rs/argon2</code>	2.2	Memory 19 MiB, 2 iterations, parallelism 1 (the OWASP baseline)
Email	Resend, React Email	6.30, 6.11	
Images	sharp, Cloudflare R2 + CDN	0.35	Covers and avatars
Logging and errors	pino, Sentry (<code>@sentry/node</code> , <code>@sentry/react-router</code>)	10.3, 11.1	
Lint and format	Biome	2.5	One tool instead of ESLint + Prettier
Testing	Vitest, Playwright	5.0, 1.63	See section 12

Key decisions

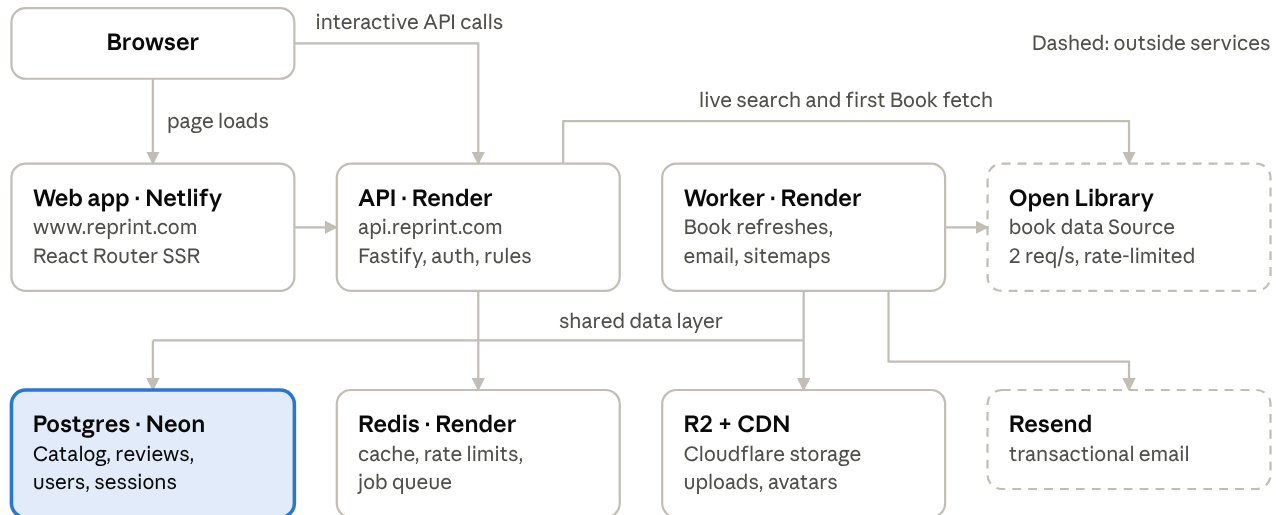
- **Server-rendered web app, separate API.** The web app's loaders call the API over HTTPS. Browsers call the API directly for interactive changes. Keeping the API separate means it can later serve a mobile app.
- **Sessions in a cookie, stored in Postgres.** The cookie is named `rp_session`. It sets `Domain=reprint.com`, so both `www` and `api` receive it, plus `HttpOnly`, `Secure`, and `SameSite=Lax`. It holds a random 256-bit token. The database stores only a SHA-256

hash of the token. When the web server renders a page, it forwards the incoming cookie to the API.

- **Worker process.** Background jobs run in the same API codebase but as a separate Render worker service (`node dist/worker.js`), so heavy jobs never slow down requests.

The diagram below shows how the pieces connect.

Pages come from the RePrint Catalog; only search goes live to a Source



RePrint architecture · 2 apps, 1 worker, 3 data stores, 2 outside services

The API and the worker share one data layer. Only live search, first Book views, and the worker's refreshes call Open Library, all through one shared rate limiter and cache. Browsers load book covers from Open Library's image server, and uploads and avatars from our CDN.

9. Data model

The database has about 30 tables in three groups: Catalog, community, and accounts/admin.

Conventions for every table:

- Primary keys are UUIDv7.
- Timestamps are `timestamptz`, stored in UTC.
- Only accounts use soft deletion.
- Every foreign key has an index.

- The full schema lives in `packages/db` as Drizzle definitions. Migrations are generated by drizzle-kit and reviewed in pull requests.

Table	Purpose	Key columns and constraints
<code>books</code>	A Book	<code>slug</code> unique; <code>title</code> , <code>subtitle</code> , <code>description</code> , <code>first_published_year</code> , <code>original_language</code> , <code>primary_edition_id</code> , <code>cover_id</code> ; <code>field_origins</code> jsonb (Source or admin, and when, for each field); <code>locked_fields</code> text[]; <code>search_vector</code> tsvector (GIN index); cached aggregates <code>review_count</code> , <code>rating_sum</code> , <code>rating_counts</code> int[5]
<code>editions</code>	An Edition	<code>book_id</code> ; <code>isbn_13</code> unique when present; <code>format</code> (hardcover, paperback, ebook, audiobook, unknown); <code>language</code> ; <code>publisher_name</code> , <code>published_date</code> , <code>page_count</code> , <code>cover_id</code> , <code>field_origins</code>
<code>authors</code>	An Author	<code>slug</code> unique; <code>name</code> , <code>alternate_names</code> text[], <code>bio</code> , <code>birth_date</code> , <code>death_date</code> , <code>photo_id</code> , <code>field_origins</code>
<code>contributions</code>	Links an Author to a Book, with a role	(<code>book_id</code> , <code>author_id</code> , <code>role</code>) primary key; <code>role</code> (author, co_author, translator, illustrator, editor, narrator, other); <code>position</code>
<code>series</code> , <code>book_series</code>	Series and their Books	<code>series.slug</code> unique; <code>book_series.position</code> numeric (can be empty)

Table	Purpose	Key columns and constraints
<code>genres</code> , <code>book_genres</code>	The curated Genre list	<code>genres.slug</code> unique; <code>parent_id</code> ; <code>featured</code> ; <code>book_genres.origin</code> (mapping or admin)
<code>subjects</code> , <code>book_subjects</code>	Raw Source subjects	<code>subjects.label</code> unique (case-insensitive)
<code>subject_genre_rules</code>	Maps Subjects to Genres	<code>pattern</code> , <code>genre_id</code> , <code>priority</code>
<code>source_links</code>	Connects a Catalog record to its ID at a Source	<code>entity_type</code> , <code>entity_id</code> , <code>source</code> , <code>source_id</code> ; unique (<code>source</code> , <code>entity_type</code> , <code>source_id</code>)
<code>source_records</code>	Raw Source responses, deleted after 30 days	<code>source</code> , <code>source_id</code> , <code>payload</code> <code>jsonb</code> , <code>fetches_at</code>
<code>covers</code>	Cover and author images	<code>origin</code> (open_library, upload, and future Sources); <code>origin_ref</code> (for example, a cover ID); <code>r2_key</code> (uploads only); <code>width</code> , <code>height</code>
<code>merge_candidates</code>	Possible duplicate Books for an admin to check	<code>book_a_id</code> , <code>book_b_id</code> , <code>reason</code> , <code>status</code>
<code>users</code>	Accounts	<code>email</code> unique (case-insensitive, <code>citext</code>); <code>username</code> unique (<code>citext</code>); <code>password_hash</code> ; <code>email_verified_at</code> ; <code>display_name</code> , <code>bio</code> , <code>avatar_id</code> ; <code>library_public</code> ; <code>status</code> (active, suspended, deleted); <code>suspended_until</code> ; <code>deleted_at</code>

Table	Purpose	Key columns and constraints
<code>roles</code> , <code>permissions</code> , <code>role_permissions</code> , <code>user_roles</code>	Access control	Seeded with Member, Moderator, and Admin
<code>sessions</code>	Login sessions	<code>token_hash</code> unique; <code>user_id</code> ; <code>expires_at</code> ; <code>last_seen_at</code> ; <code>ip</code> ; <code>user_agent</code>
<code>auth_tokens</code>	Email verification, password reset, and email change links	<code>token_hash</code> unique; <code>purpose</code> ; <code>expires_at</code> ; <code>used_at</code>
<code>reviews</code>	A Review	unique (<code>user_id</code> , <code>book_id</code>); <code>rating</code> 1 to 5 (check constraint); <code>headline</code> , <code>body</code> , <code>has_spoilers</code> , <code>edition_id</code> ; <code>status</code> (pending, approved, rejected, unpublished); <code>helpful_count</code> ; <code>submitted_at</code> , <code>decided_at</code>
<code>review_versions</code>	Every submitted version of a Review	<code>review_id</code> , <code>version</code> , full content, <code>status</code> , <code>decided_by</code> , <code>decision_reason</code>
<code>review_claims</code>	Moderator claims on a queue item	<code>review_id</code> primary key; <code>moderator_id</code> ; <code>expires_at</code>
<code>helpful_votes</code>	Helpful votes	(<code>review_id</code> , <code>user_id</code>) primary key
<code>review_reports</code>	Reports	unique (<code>review_id</code> , <code>reporter_id</code>); <code>reason</code> , <code>note</code> , <code>status</code> , <code>resolved_by</code> , <code>resolution</code>
<code>shelf_entries</code>	Library	unique (<code>user_id</code> , <code>book_id</code>); <code>shelf</code> (want_to_read, reading, read); <code>added_at</code> , <code>updated_at</code>
<code>notifications</code>	In-app notifications	<code>user_id</code> , <code>type</code> , <code>data</code> jsonb, <code>read_at</code>

Table	Purpose	Key columns and constraints
<code>audit_log</code>	Record of elevated actions	<code>actor_id</code> , <code>action</code> , <code>target_type</code> , <code>target_id</code> , <code>before_jsonb</code> , <code>after_jsonb</code> , <code>ip</code> , <code>created_at</code> . The app's database role can only insert rows
<code>featured_items</code>	Discover page picks	<code>kind</code> (genre, review), <code>ref_id</code> , <code>position</code>

Keeping ratings in sync. A Book's cached review count and rating totals are updated in the same database transaction as any change to a review's status. A nightly job recalculates them from scratch and alerts if they don't match.

10. API design

The API is a versioned JSON REST API at `https://api.reprint.com/v1`. It speaks only RePrint's vocabulary (section 5).

Conventions

- **Validation:** Every request body, query, and URL parameter is checked with a Zod schema from `packages/shared`. Responses are serialized through schemas too, so internal fields can never leak.
- **Errors:** Returned in the standard Problem Details format (RFC 9457): `{ type, title, status, detail, errors?: [{ path, message }] }`.
- **Pagination:**
 - Public lists use `?page=&pageSize=` (maximum 50), matching URLs people can share.
 - Admin queues use cursor pagination (`?cursor=`).
- **Auth:** The `rp_session` cookie. Any change request (not GET) must include an `Origin` header from `https://www.reprint.com` (or the preview/staging web origin); otherwise it is rejected with 403.
- **API documentation:** An OpenAPI 3.1 spec is generated from the Zod schemas and published at `/v1/docs` outside production.
- **Caching:** Public GET responses set `Cache-Control` with `stale-while-revalidate` and an `ETag`.

Endpoints

Area	Endpoints
Auth	POST /auth/register , /auth/login , /auth/logout , /auth/logout-all , /auth/verify-email , /auth/resend- verification , /auth/forgot-password , /auth/reset-password · GET /auth/session
Me	GET/PATCH /me · POST /me/email , /me/password , /me/avatar · GET/DELETE /me/sessions/:id · DELETE /me · GET /me/notifications , POST /me/notifications/read
Catalog	GET /search? q=&type=books authors&genre=&language=&decad e=&minRating=&sort=&page= · GET /search/suggest?q= · GET /books/:slug · GET /books/:slug/editions · GET /authors/:slug · GET /genres , /genres/:slug · GET /series/:slug · GET /discover
Reviews	GET /books/:slug/reviews? sort=&rating=&page= · GET/PUT/DELETE /books/:slug/my-review · POST/DELETE /reviews/:id/helpful · POST /reviews/:id/reports
Library	PUT/DELETE /books/:slug/shelf · GET /users/:username/library?shelf=&sort=&page=
Profiles	GET /users/:username · GET /users/:username/reviews
Moderation	GET /mod/reviews?cursor= · POST /mod/reviews/:id/claim · POST /mod/reviews/:id/approve , /reject , /unpublish · GET /mod/reports · POST /mod/reports/:reviewId/dismiss · GET /mod/stats

Area	Endpoints
Admin	GET /admin/users , /admin/users/:id · PUT/DELETE /admin/users/:id/roles/:role · POST /admin/users/:id/suspend , /unsuspend , /revoke-sessions , /resend-verification · PATCH /admin/books/:id · POST /admin/books/:id/refresh , /admin/books/merge · PUT /admin/featured · GET /admin/audit , /admin/audit.csv
Ops	GET /health (process is up) · GET /ready (Postgres, Redis, and queue are reachable)

Each Moderation and Admin endpoint requires the matching permission from section 4. Permissions are checked in a Fastify `preHandler` hook, never inside route code.

11. Non-functional requirements

Security

- Follow the OWASP ASVS 5.0 Level 2 checklist for authentication, sessions, and access control. The pre-launch security review signs off against it.
- Validate every input with Zod. Every query goes through Drizzle with parameters, never string-built SQL.
- React escapes output by default. `dangerouslySetInnerHTML` is banned by a lint rule.
- Security headers are set by `@fastify/helmet` and Netlify: a strict Content-Security-Policy with nonces, HSTS (with preload), X-Content-Type-Options , and Referrer-Policy: `strict-origin-when-cross-origin` .
- Rate limits are stored in Redis:

Action	Limit
Log in	10 per 15 minutes per IP; 5 per 15 minutes per account
Register	5 per hour per IP
Password reset and resend verification	3 per hour per email

Action	Limit
Review create or edit	20 per day per user
Report	20 per day per user
Other authenticated writes	120 per minute per user
Anonymous reads	300 per minute per IP

- Secrets are kept in Render and Netlify environment settings, never in the repository. Gitleaks runs in CI.
- Dependabot or Renovate is enabled. `pnpm audit` fails CI on high and critical vulnerabilities.
- Uploads are checked by their actual file contents (not the file extension), limited to 5 MB, and re-encoded with sharp, which also removes EXIF data.

Performance targets (production, measured at p95)

Measure	Target
API read endpoints	200 ms or less
API write endpoints	400 ms or less
Search	300 ms or less
Book page, Largest Contentful Paint	2.5 s or less on a mid-range phone over 4G
Interaction to Next Paint	200 ms or less
Cumulative Layout Shift	0.1 or less
Load test (before launch)	200 requests per second with a mixed read-heavy workload for 10 minutes, less than 1% errors

SEO

- Server-rendered HTML with a canonical URL and meta description on every page.
- Sitemaps are split into chunks and rebuilt nightly.
- `schema.org` `Book`, `Review`, `Person`, and `BreadcrumbList` data.
- Unverified and admin pages are marked `noindex`.

Accessibility

- Meet WCAG 2.2 AA on every page. axe checks run in CI via Playwright, and a manual screen reader pass (VoiceOver and NVDA) happens before launch.
- The star rating input works with the keyboard as a radio group.
- Spoiler toggles and the rating distribution chart have text alternatives.

Privacy

- Collect the minimum data needed. Analytics come from Plausible or self-hosted Umami, which use no cookies.
- IP addresses in sessions and the audit log are kept for 90 days.
- Users can download their data as JSON from settings.
- The Privacy Policy covers GDPR and CCPA rights.

Reliability and observability

- Uptime target: 99.9% monthly, watched by an outside uptime monitor on `/ready` and the home page.
- Structured JSON logs from pino, each with a request ID shared by the web app and API.
- Sentry for errors and performance tracing on both apps, with source maps uploaded in CI.
- Alerts go to email and Slack when:
 - the error rate exceeds 2%
 - p95 latency is over target for 10 minutes
 - the job queue is stuck (more than 1,000 waiting or the oldest job is 15 minutes old)
 - the Source's circuit breaker opens
 - the oldest pending review is more than 48 hours old
- Backups: Neon point-in-time restore (7 days or more), plus a nightly logical dump to R2 kept for 30 days. A restore drill is done before launch and every quarter after that.

12. Testing and CI/CD

Every pull request must pass the full pipeline before it can merge to `main`. Every merge to `main` deploys to staging automatically.

Test layers

Layer	Tool	Covers	Bar
Unit	Vitest	Domain logic: permission checks, weighted rating, slug generation, adapter translation, validation schemas	90% line coverage in packages/shared and domain modules
Adapter contract	Vitest + recorded fixtures	Each Source adapter turns real saved Open Library responses (works, editions, authors, search, and edge cases like missing covers) into valid domain types	Every adapter passes the shared contract suite
API integration	Vitest + Testcontainers (Postgres 18, Redis)	Every endpoint: success cases, validation failures, permission denials, rate limits, and review status changes, against a real database	Every endpoint has at least one allowed and one denied test
Component	Vitest + Testing Library	Forms, the star rating input, spoiler toggle, shelf selector, moderation queue	Main interactive components

Layer	Tool	Covers	Bar
End-to-end	Playwright (Chromium, WebKit, mobile viewport)	Register, verify, and log in; password reset; search to book page; write, edit, and delete a review; moderate a review; report and unpublish; shelve and view a library; admin assigns a role and suspends a user	Runs on every PR against a preview environment
Accessibility	<code>@axe-core/playwright</code>	Every page visited by the end-to-end tests	Zero serious or critical issues
Load	k6	Section 11 load target	Before launch and before major releases

End-to-end tests use a mocked email inbox (Resend's test mode, or a local capture server in CI) and a stubbed Source, so tests are repeatable.

Pipeline (GitHub Actions)

1. **Install:** `pnpm install --frozen-lockfile`, with the Turborepo remote cache.
2. **Check:** Biome lint and format, `tsc` type check, migration drift check (the Drizzle schema must match the migrations), Gitleaks, and `pnpm audit`.
3. **Test:** Unit, contract, component, and API integration tests, run in parallel.
4. **Build:** Both apps, plus the OpenAPI spec (the build fails if the spec changes without being committed).
5. **Preview:** A Netlify deploy preview for the web app, plus a Render preview for the API with its own Neon database branch. Playwright and axe then run against the preview.
6. **Merge to `main`:** Migrations run against staging, then staging deploys, then smoke tests run.
7. **Release:** Tagging a version promotes to production. It runs migrations with a pre-deploy command, then deploys the API and worker, then the web app, then smoke tests. Sentry release markers and source maps are uploaded.

Migration rule: Migrations must stay compatible with the previous version of the code (expand, then contract), so a deploy can be rolled back without rolling back the database.

13. Environments and deployment

There are four environments. Each has its own database and secrets. Production and staging are set up identically, so staging is a reliable test of production.

Environment	Web	API and worker	Database	Email	Source calls
Local	<code>localhost:5173</code>	<code>localhost:3000</code>	Docker Postgres 18 + Redis via <code>docker</code> <code>compose</code>	Local capture (Mailpit)	Recorded fixtures by default; live Open Library by opting in
Preview (per PR)	Netlify deploy preview	Render preview	Neon branch from staging	Resend test mode	Stubbed
Staging	<code>staging.reprint.com</code>	<code>api.staging.reprint.com</code>	Neon staging	Resend (sandbox domain)	Live, rate limit 1 per second
Production	<code>www.reprint.com</code>	<code>api.reprint.com</code>	Neon production	Resend (<code>mail.reprint.com</code> , with SPF, DKIM, and DMARC set up)	Live, rate limit 2 per second

Hosting notes

- **Render:** The API runs as a paid web service (no sleeping) with at least 2 instances behind Render's load balancer. The worker is a separate background worker service. Redis uses Render Key Value in the same region.
- **Region:** Neon, Render, and Redis all run in the same US East region, to keep database round trips short.
- **Netlify:** Hosts the React Router server-rendered app via `@netlify/vite-plugin-react-router` (4.0). Static assets are cached permanently since their filenames change with every build.
- **DNS and CDN:** DNS is managed in Cloudflare. `img.reprint.com` serves images from R2 through Cloudflare's CDN.
- **Local setup:** `pnpm dev` starts everything. `pnpm db:reset` recreates the database and loads sample data: about 500 Books, 50 users, and reviews in every status.

14. Open questions and decisions

Open questions

- ☐ **Launch content:** The Catalog starts empty, so Discover will be sparse on day one. Should we invite a small group of early reviewers for a private beta, to fill in books and reviews before the public launch?
- ☐ **Edited reviews:** While an edited review waits for approval again, should the previously approved version stay visible? Current spec: no, it is hidden (as decided). Keeping it visible would avoid books temporarily losing reviews.
- ☐ **Deleted accounts:** Should a deleted account's approved reviews be erased (current spec) or kept as "Deleted reader"? Keeping them preserves book pages but needs a clear line in the Privacy Policy.
- ☐ **Second Source at launch:** Should we ask Hardcover now whether its data can be stored? If yes, a Hardcover adapter could fill in series, roles, genres, and descriptions before launch. Google Books is ruled out as a Catalog Source by its terms (section 6).
- ☐ **Genre list:** Who writes the initial list of about 40 Genres and the rules that map Subjects to them? I can draft both from the most common Open Library subjects, based on a sample of search results, for your review.
- ☐ **Brand:** Is `reprint.com` actually secured? Every domain in this document is a placeholder until it is.
- ☐ **Moderation staffing:** Who will moderate at launch, and how fast should reviews be decided? The 48-hour alert assumes at least daily coverage.

Decisions made

Decision	Choice	Reason
Scope	One v1 launch with every feature in this document	Owner requirement
Book data	RePrint Catalog that stores Books as they're used, plus live Source search with caching; Open Library first; no bulk import	Own vocabulary; small storage cost; pages never wait on a Source; stays within Open Library's usage guidelines
Reviews	One per Member per Book; every review moderated; edits go back to Pending; rejection reason optional	Owner requirement

Decision	Choice	Reason
Reviewing	Doesn't require the Book to be in the library	Owner requirement
Library	One shelf per Book; no reading dates	Libraries are secondary
Access control	Roles and permissions tables; Member, Moderator, Admin	Room to add roles later
Sign-in	Postgres sessions in an httpOnly cookie on <code>reprint.com</code> ; Argon2id	Sessions can be revoked; <code>www</code> and <code>api</code> are the same site
Email verification	Required before reviewing, voting, or reporting	Reduces spam
Web app	React Router 8 framework mode, server-rendered on Netlify	Book pages need to be indexed by search engines
API	Fastify 5 on Render with a separate worker	Can later serve a mobile app; heavy jobs don't slow requests
Repository	pnpm + Turborepo monorepo	Zod schemas shared between web and API
Quality	Full test suite, CI/CD, preview environments	Production-ready from day one